Unit 7
Pipelining

ECE-GY 6463: ADVANCED HARDWARE DESIGN

PROFS. SUNDEEP RANGAN, SIDDHARTH GARG

Learning Objectives

Skill	You will be able to
Pipeline a sequential module	Insert registers between stages, keep every path aligned, and find the II, the latency and the total time
Perform static timing analysis	Find each stage's critical path, the clock it allows, and each block's utilization
Write pipelined SystemVerilog	Write the registers of a pipeline, and trace its values cycle by cycle
Read HLS synthesis reports	Read the II, iteration latency and resources of a pipelined loop, and predict its time for any trip count
Find the II a loop-carried dependency allows	Trace the values one iteration passes to the next, and break a recurrence with interleaved partial sums

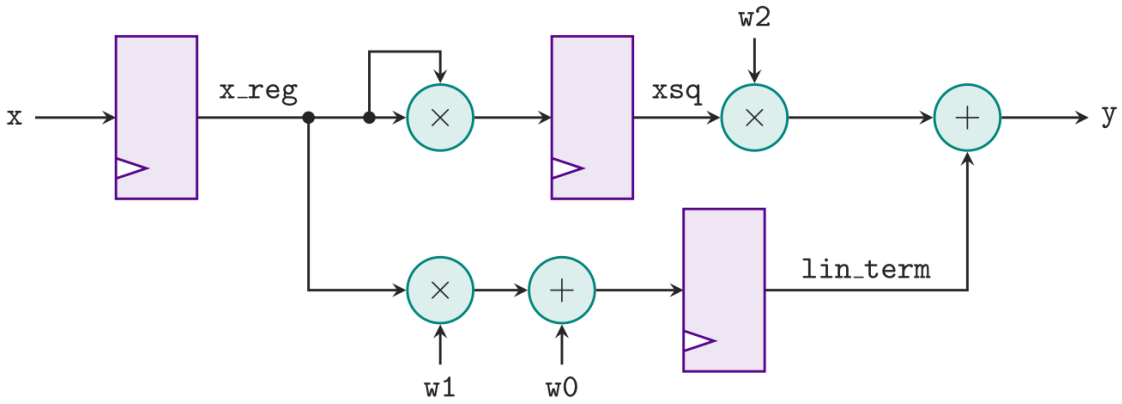
Each skill is assessed by problems in the LLM grader. Loop unrolling and memory partitioning are unit 8.

Outline

- Pipelining in SystemVerilog
- Loop Pipelining in Vitis HLS
- Limits on the Initiation Interval

Recap: Pipelining the Quadratic

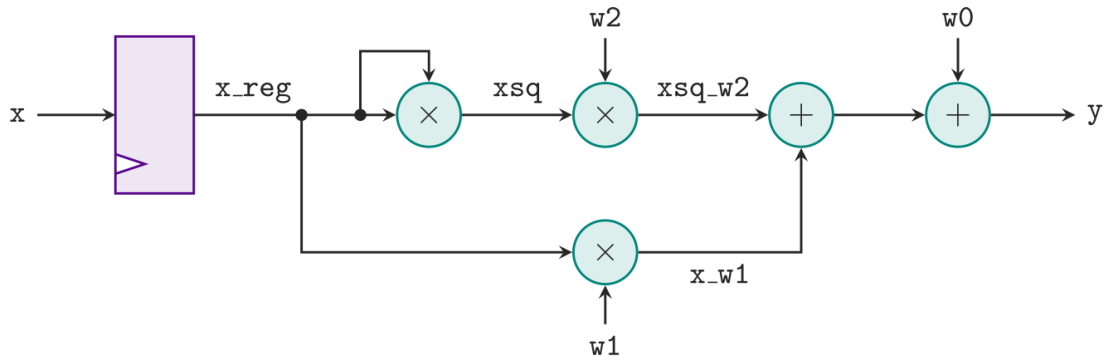
- Unit 6 split $y = w_2x^2 + w_1x + w_0$ over **two cycles**
  - Register xsq and lin_term between the stages
  - Register 1 ns, multiply 4 ns, add 1.5 ns
- The clock nearly doubles; the latency grows by 1 ns
- This unit: **why** it works, how to pipeline any circuit, and how Vitis HLS does it for loops



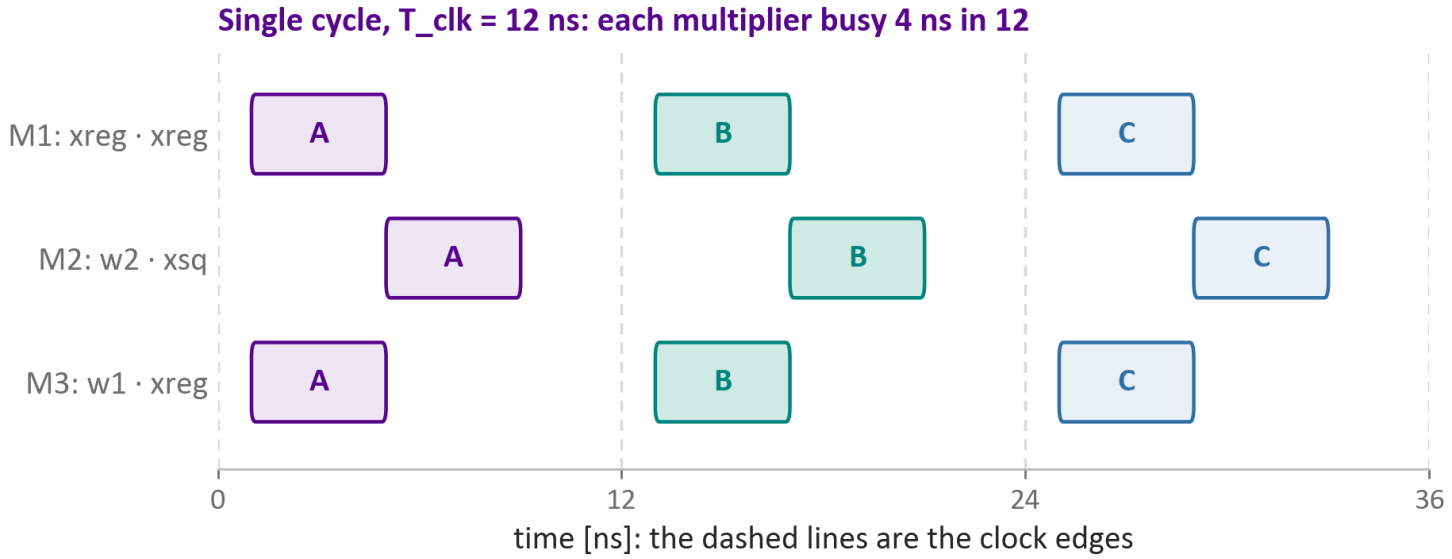
	Single cycle	Two cycles
Critical path	12 ns	6.5 ns
Max clock	≈ 83 MHz	≈ 154 MHz
Latency	12 ns	13 ns

Multiplier Utilization

- When is each multiplier actually **busy**?
- Single cycle, T_{clk} = 12 ns
  - M1 and M3: busy from 1 ns, when xreg is stable, to 5
  - M2 waits for xsq: busy from 5 to 9
  - The rest of the cycle each one is **idle**, holding its output
- **Utilization** = busy time / T_{clk}
  - Here 4 / 12 ≈ **33%** for each multiplier

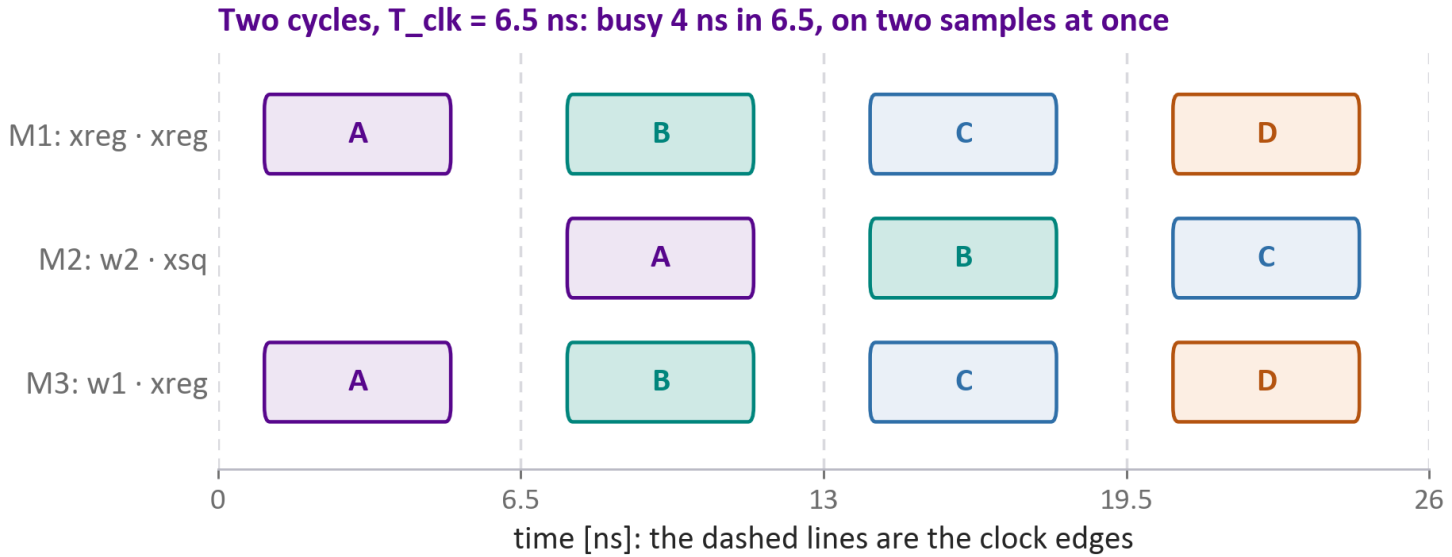


Plotting the Utilization



- One bar per multiplication, labelled with its input sample
- Each multiplier works on **one sample** per cycle, briefly
- M2 cannot start until M1 is done
- Shortening the clock is the only way to use them more

Utilization with Two Cycles



- T_{clk} = 6.5 ns: each multiplier busy 4 ns in 6.5, **62%**
- M2 works on sample A **while** M1 and M3 work on B
- Two samples are in flight at once: a **pipeline**
- More throughput from the same three multipliers

Comparison

	Single cycle	Two cycles
Clock period	12 ns	6.5 ns
Throughput	≈ 83 M samples/s	≈ 154 M samples/s
Latency	12 ns	13 ns
Multiplier utilization	4 / 12 ≈ 33%	4 / 6.5 ≈ 62%
Registers	xreg	xreg, xsq, lin_term

Pipelining: higher throughput through better utilization, for a little latency and a few registers.

Multiplier Utilization

PRACTICE

A circuit computes $y = (a \cdot x + b) \cdot x + c$ with two multipliers and two adders. x is registered as xreg, and y is the last stage's combinational output. Ignore the setup time.

For each design, find T_{clk}, the throughput, each multiplier's and each adder's utilization, and the latency from the edge that captures x until y is stable.

(a) One stage: all four operations in one cycle

(b) Two stages: registers on $z = a \cdot x + b$ and on a delayed xreg

(c) Four stages: a register after every multiplier and adder

The delays

Block	Delay
Register	1 ns
Multiplier	4 ns
Adder	1.5 ns

Also in the LLM grader: Unit 7 › Multiplier utilization

Multiplier Utilization

SOLUTION

Design	T_{clk}	Throughput	Multiplier	Adder	Latency
(a) one stage	12 ns	83 M/s	33%	12.5%	12 ns
(b) two stages	6.5 ns	154 M/s	62%	23%	13 ns
(c) four stages	5 ns	200 M/s	80%	30%	17.5 ns

- T_{clk}: the slowest stage, register delay included
  - (b): 1 + 4 + 1.5 = 6.5 ns, both stages
  - (c): multiplier stages 1 + 4 = 5 ns, adder stages 2.5 ns
- Latency: full cycles, then the last stage: 3 × 5 + 2.5 = 17.5 ns

Common error

Leaving out the 1 ns register delay: 5.5 ns and 182 MHz for (b).

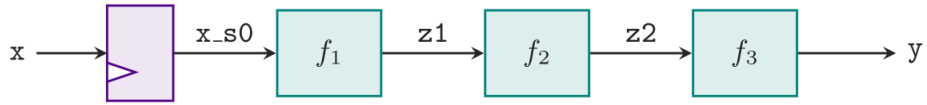
In (c) the short adder stages idle 70% of the time: the stages are unbalanced.

Also in the LLM grader: Unit 7 › Multiplier utilization

Pipelining a Series

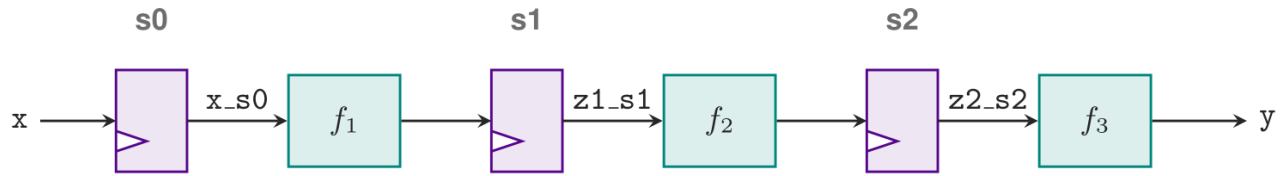
ONE CYCLE

```
always_ff @(posedge clk)
    x_s0 <= x;
always_comb begin
    z1 = f1(x_s0);
    z2 = f2(z1);
    y  = f3(z2);
end
```



THREE STAGES

```
always_ff @(posedge clk) begin
    x_s0  <= x;
    z1_s1 <= f1(x_s0);
    z2_s2 <= f2(z1_s1);
end
always_comb
    y = f3(z2_s2);
```



A register after each stage. The suffix _s0, _s1, _s2 names the stage whose register holds the value.

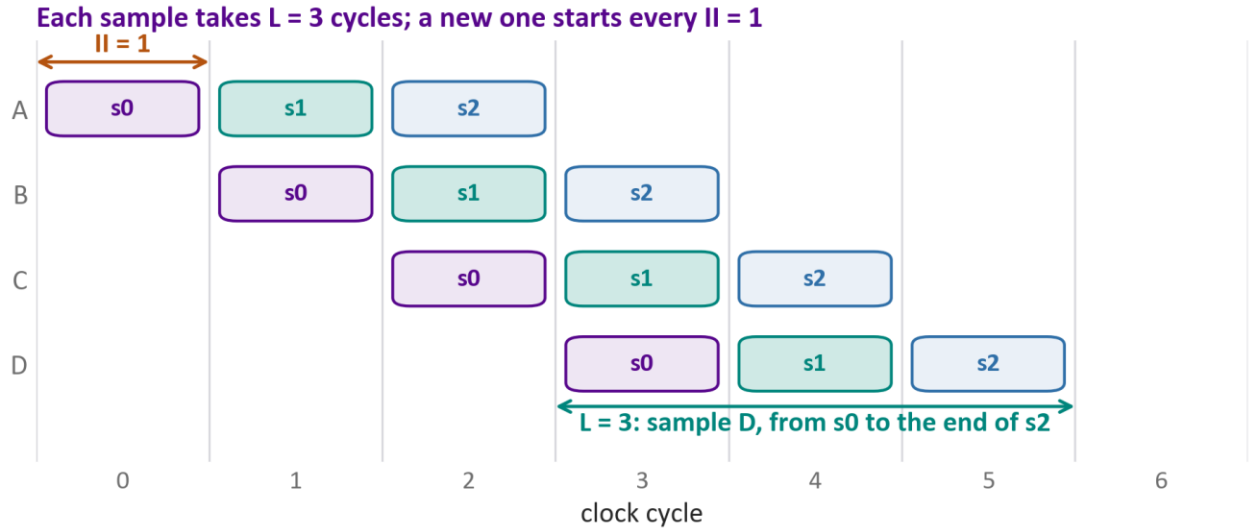
Cycle-by-Cycle Timing

Cycle	x	x_s0	z1_s1	z2_s2	y
0	A	?	?	?	?
1	B	A	?	?	?
2	C	B	$f_1(A)$	?	?
3	D	C	$f_1(B)$	$f_2(f_1(A))$	$f_3(f_2(f_1(A)))$
4		D	$f_1(C)$	$f_2(f_1(B))$	$f_3(f_2(f_1(B)))$
5			$f_1(D)$	$f_2(f_1(C))$	$f_3(f_2(f_1(C)))$
6				$f_2(f_1(D))$	$f_3(f_2(f_1(D)))$

- Inputs A, B, C, D arrive one per cycle
- Each value moves **one stage per cycle**
- $y = f_3(f_2(f_1(A)))$ appears in cycle 3, three cycles after A
- From cycle 3 on, a new output **every cycle**

Pipeline Terminology

- **Initiation interval (II)**
  - Cycles between the starts of two inputs
- **Latency (L)**
  - Cycles from an input to its output
  - The number of register stages it crosses
- **Number of samples (n)**
- **Total processing time**: T = (n − 1)·II + L cycles
  - The first output after L, then one every II



Pipeline Cycle Values

PRACTICE

The pipeline on the right has input x and output y.

The inputs on cycles 0, 1 and 2 are 3, 6 and 12, and unknown (?) after. Every register is unknown at cycle 0.

Write the values of x_s0, z1_s1, z2_s2 and y for cycles 0 to 5.

```
always_ff @(posedge clk) begin
    x_s0  <= x;
    z1_s1 <= 10*x_s0;
    if (z1_s1 > 100)
        z2_s2 <= 150 - (z1_s1>>>1);
    else
        z2_s2 <= z1_s1;
end
always_comb
    y = (3*z2_s2>>>2);
```

Also in the LLM grader: Unit 7 › Pipeline cycle values

Pipeline Cycle Values

SOLUTION

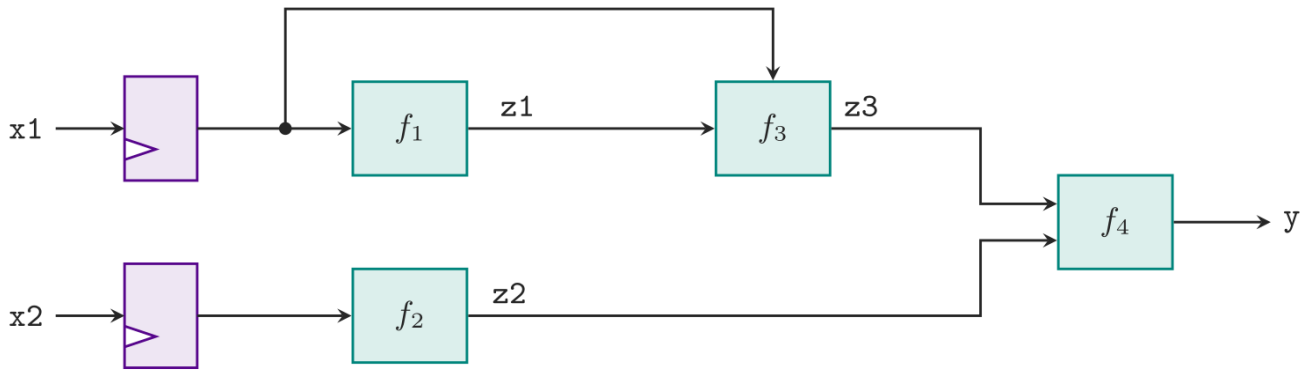
Cycle	x	x_s0	z1_s1	z2_s2	y
0	3	?	?	?	?
1	6	3	?	?	?
2	12	6	30	?	?
3	?	12	60	30	22
4	?	?	120	60	45
5	?	?	?	90	67

- Each register takes the value computed from the stage before, **one cycle later**
- y is combinational: same cycle as z2_s2
  - Cycle 3: y = (3·30) >>> 2 = 90 >>> 2 = 22
  - Cycle 5: z1_s1 was 120 > 100, so z2_s2 = 150 − 60 = 90 and y = 270 >>> 2 = 67
- After cycle 2 the unknown inputs flow through, one stage per cycle

Also in the LLM grader: Unit 7 › Pipeline cycle values

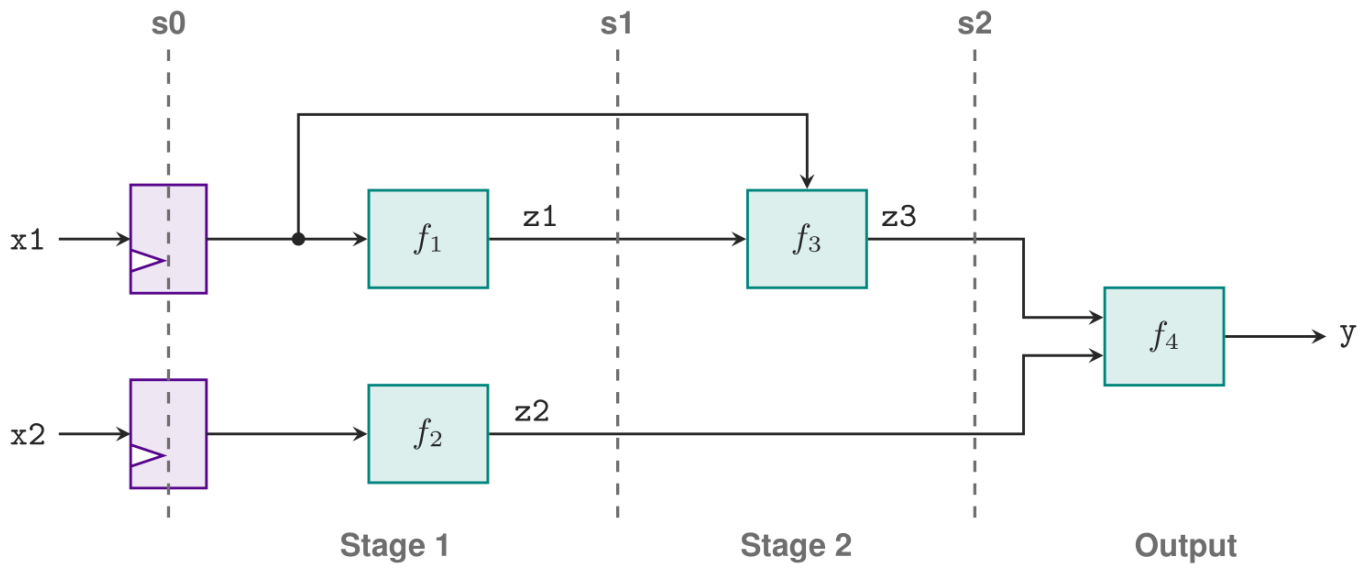
Pipelining with Multiple Paths

- Now a circuit with **several paths**
  - x1 reaches f_3 directly and through f_1
  - x2 reaches f_4 through f_2 alone
- How do we pipeline it?



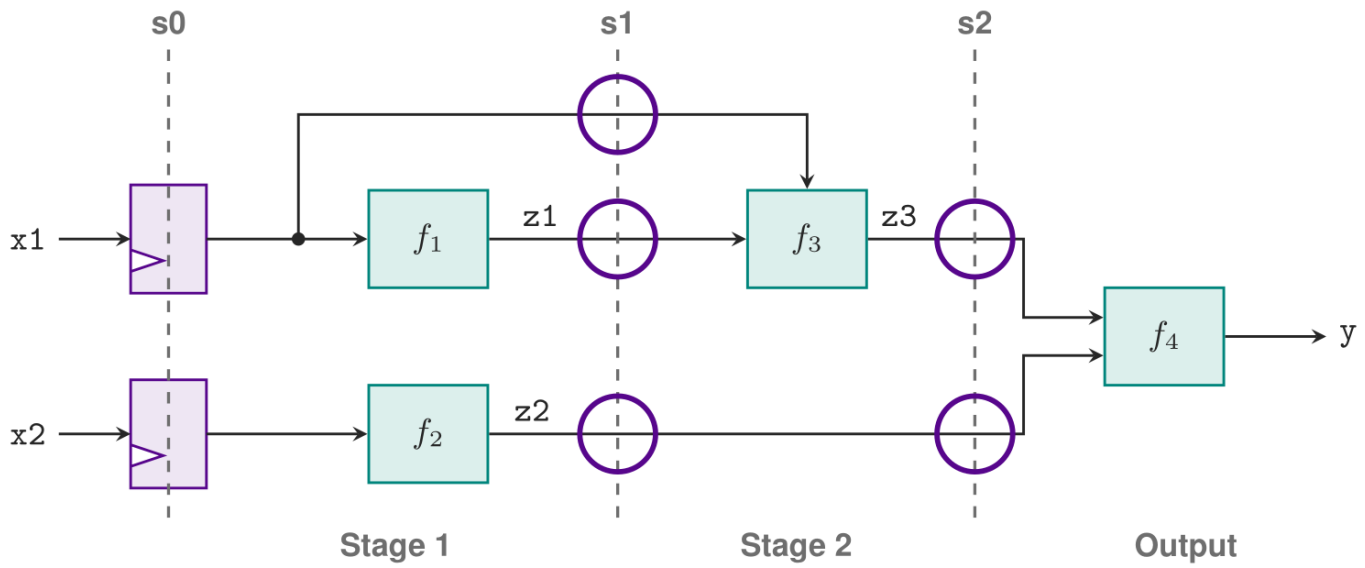

```
always_ff @(posedge clk) begin
    x1_reg <= x1;
    x2_reg <= x2;
end
always_comb begin
    z1 = f1(x1_reg);
    z2 = f2(x2_reg);
    z3 = f3(z1, x1_reg);
    y  = f4(z2, z3);
end
```

Multi-Path Pipelining: Draw the Stages



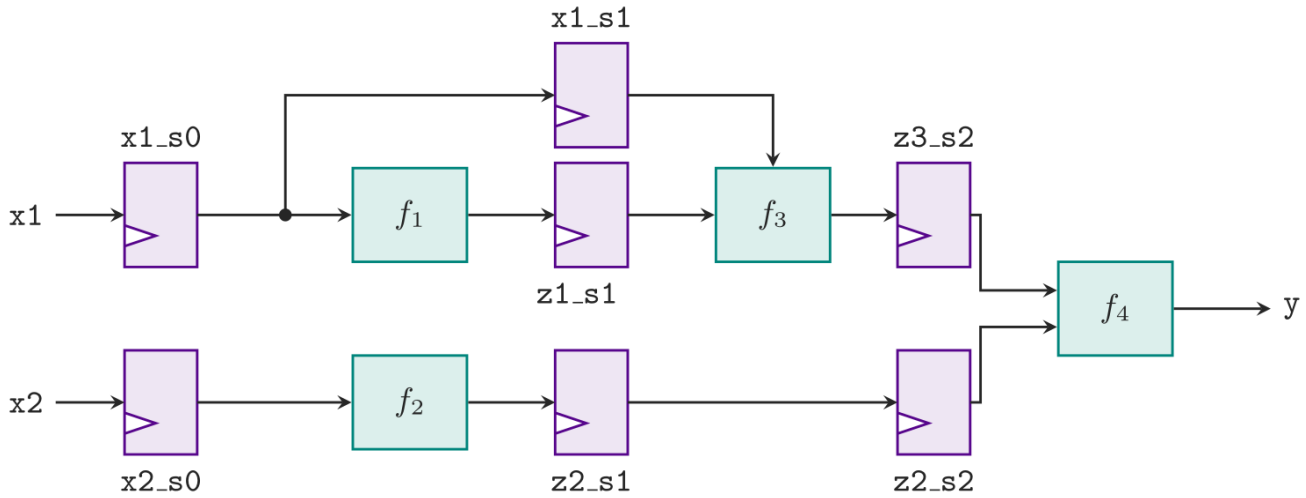
- Draw a **cut line** where each stage's registers will go
  - s0: the input registers, already there
  - s1 after f_1, f_2; s2 after f_3
- Each stage must fit in one clock period
- After the last line the output is combinational

Multi-Path Pipelining: Place the Registers



- A register **wherever a wire crosses a line**
- Even on a wire that only **passes through** a stage
  - x1 to f_3 crosses s1: it needs x1_s1
  - z2 crosses s1 and s2: it needs two
- Otherwise f_3 and f_4 would combine values of **different samples**

Final Pipelined Diagram and Code




```
always_ff @(posedge clk) begin
    x1_s0 <= x1;
    x2_s0 <= x2;
    z1_s1 <= f1(x1_s0);
    z2_s1 <= f2(x2_s0);
    x1_s1 <= x1_s0;
    z3_s2 <= f3(z1_s1, x1_s1);
    z2_s2 <= z2_s1;
end
always_comb
    y = f4(z2_s2, z3_s2);
```

Homework in the LLM grader: Unit 7 › Pipelined maximum

Two-Cycle Design

PRACTICE

The register outputs are stable at the rising edge; the delays are in the comments.

(a) Draw the block diagram from x1 and x2 to y, each block labelled with its delay, and mark when each signal is stable

(b) From the diagram, find the critical path and its delay

(c) Rewrite the code with pipeline registers on z1, z2 and xreg1, and redraw the diagram

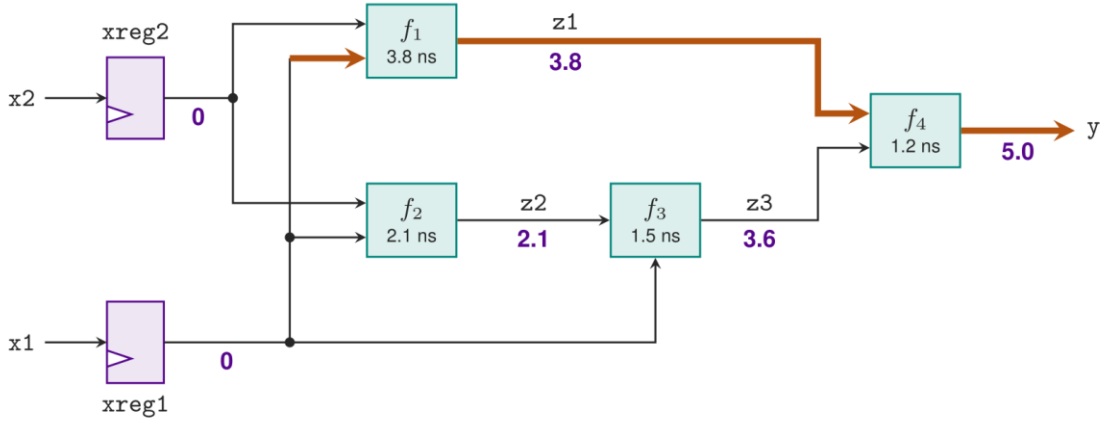
(d) Find the critical path of the two-cycle design

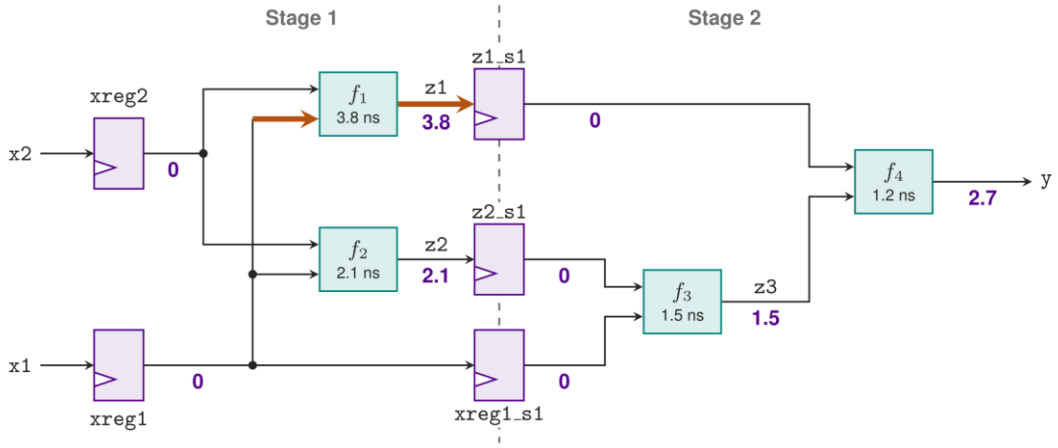
```
always_ff @(posedge clk) begin
    xreg1 <= x1;
    xreg2 <= x2;
end
always_comb begin
    z1 = f1(xreg1, xreg2);  // 3.8 ns
    z2 = f2(xreg1, xreg2);  // 2.1 ns
    z3 = f3(xreg1, z2);     // 1.5 ns
    y  = f4(z1, z3);        // 1.2 ns
end
```

Also in the LLM grader: Unit 7 › Two Cycle Design

Two-Cycle Design

SOLUTION





(b) Through f_1 and f_4: 3.8 + 1.2 = **5.0 ns**. Through f_2, f_3, f_4: 4.8 ns

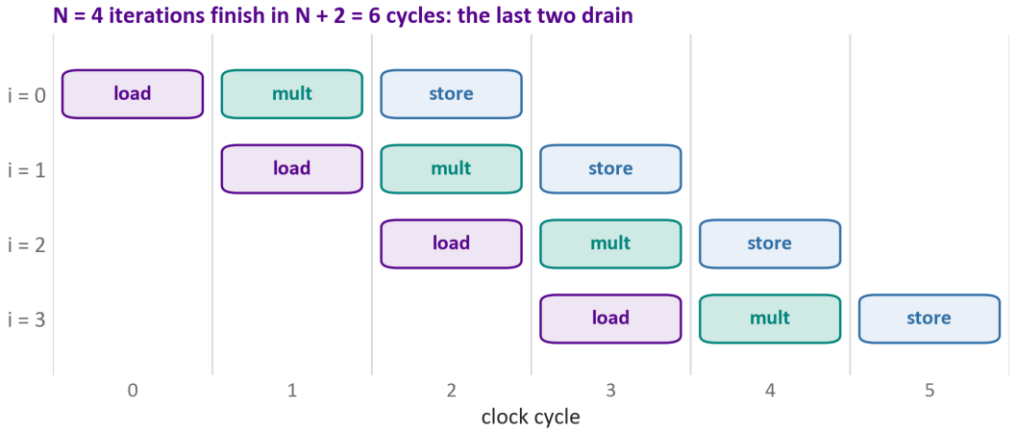
(c) Register z1, z2 and xreg1; f_3 and f_4 in stage 2

(d) Stage 1, through f_1: **3.8 ns**. Stage 2: 1.5 + 1.2 = 2.7 ns. The clock is set by the slower stage, not the sum

Also in the LLM grader: Unit 7 › Two Cycle Design

Memory Accesses

- A memory read takes at least **one cycle**
  - The address goes in on one edge, the word comes out after it
- So load, compute and store are separate stages
- The pipeline **drains**: y[i] = x[i]·w[i] needs N + 2 cycles




```
always_ff @(posedge clk) begin
    // Stage 0: read x[i] and w[i]
    if (running && (i < N)) begin
        x_s0 <= x[i];
        w_s0 <= w[i];
    end
    // Stage 1: compute the product
    if (running && (i >= 1) && (i < N+1))
        prod_s1 <= x_s0 * w_s0;
    // Stage 2: write the output
    if (running && (i >= 2) && (i < N+2))
        y[i-2] <= prod_s1;
    if (running && (i < N+2))
        i <= i + 1;
end
```

Outline

- Pipelining in SystemVerilog
- Loop Pipelining in Vitis HLS
- Limits on the Initiation Interval

Why Use Vitis HLS?

- Pipelining by hand in SystemVerilog is
  - Tedious, and easy to get wrong: every pass-through register
  - Hard to schedule well
  - Redone for each clock and each device
- In Vitis HLS, the same loop is four lines
  - `PIPELINE II=1`: start one iteration every cycle
  - The tool finds the stages, the registers and the drain
- The tool also reports what it built: II, latency, resources

THE MEMORY EXAMPLE IN VITIS HLS

```
for (int i = 0; i < N; i++) {
#pragma HLS PIPELINE II=1
    y[i] = x[i] * w[i];
}
```

The Demo Kernel: a Cubic

- $y = c_3x^3 + c_2x^2 + c_1x + c_0$ on up to 1024 samples
  - Horner's method, in FP32
- Each iteration: one load, **3 multiplies, 3 adds**, one store
  - The float cores are themselves multi-cycle
  - Hard to pipeline by hand
- x and y are block RAMs with one port: `ap_memory`
- Built twice: `PIPELINE off` and `PIPELINE II=1`

```
static float horner(const float c[4], float x) {
#pragma HLS INLINE
    float y = c[3];
    y = y * x + c[2];
    y = y * x + c[1];
    y = y * x + c[0];
    return y;
}
...
sample_loop: for (int i = 0; i < nsamp; i++) {
#pragma HLS PIPELINE II=1     // or: PIPELINE off
    y[i] = horner(c, x[i]);
}
```

The Demo: Pipelining a Loop

sdrangan.github.io/hwdesign — Demos › Loop Optimization

RUN THE WHOLE FLOW

```
cd demos/pipeline/poly
python poly_build.py --list-steps
python poly_build.py --through pipe_report
python poly_build.py --through schedule_pipe
python poly_build.py --through timing_pipe
```

WHAT TO WATCH FOR

- `synth_pipe` synthesizes the loop **without and with** the pragma
- `pipe_report` reads II, latency and resources from both reports
- `schedule_pipe` draws what happens in each cycle of one iteration
- `timing_pipe` reads the co-simulation VCD: one y every cycle

1024 samples: 30,729 cycles without the pragma, 1,063 with it. 15 DSPs instead of 5.

The pipelining this unit does by hand, done by the tool, and measured.

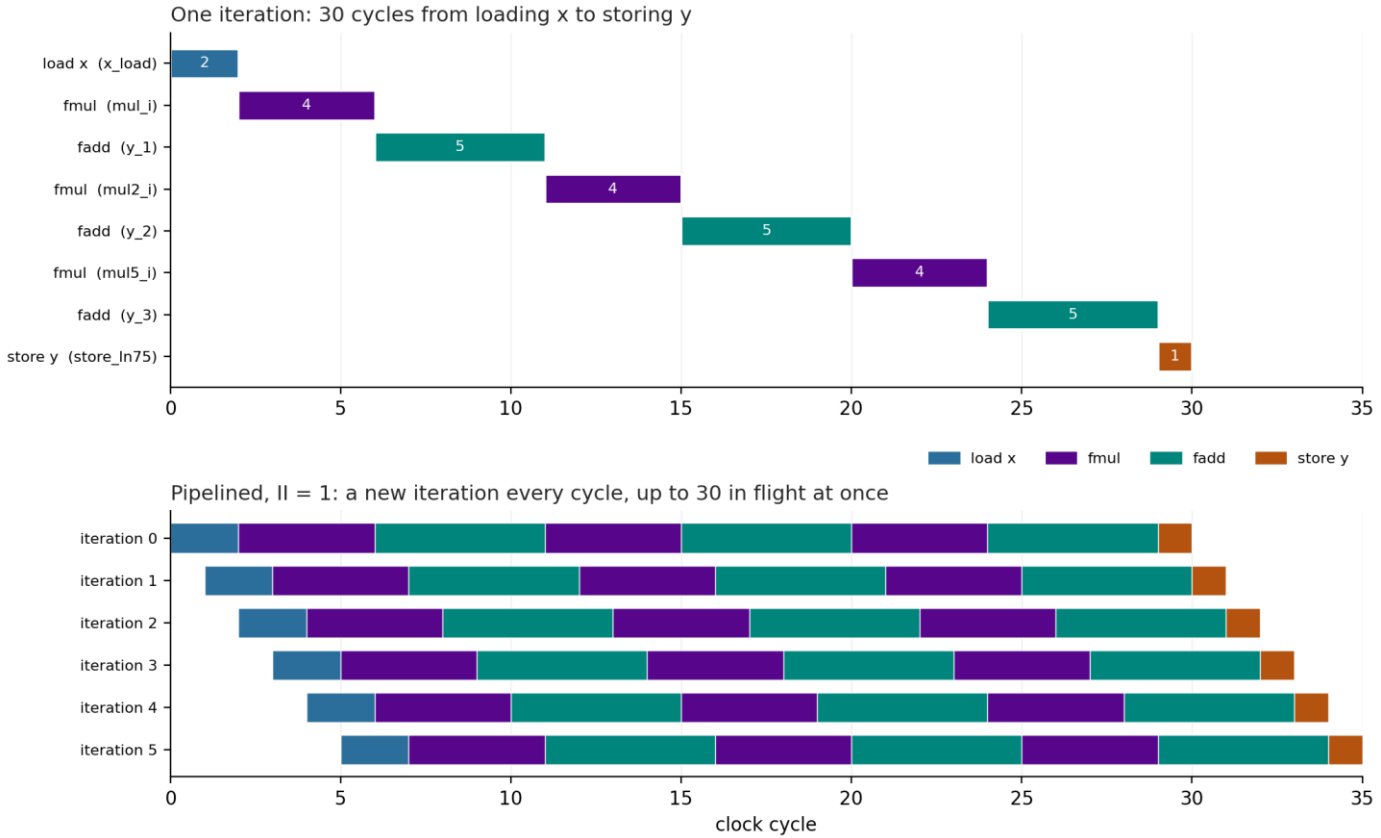
Reading the Synthesis Report

	PIPELINE off	PIPELINE II=1
Initiation interval	– (30 per iteration)	1
Iteration latency	30 cycles	30 cycles
Latency, 1024 samples	30,729 cycles	1,063 cycles
Time at 10 ns	307 µs	10.6 µs
DSP / FF / LUT	5 / 700 / 1,103	15 / 1,651 / 2,444

From synth_pipe's two csynth reports, Vitis HLS 2025.1, Zynq-7020 at 100 MHz. pipeline.md in the demo docs walks through the report.

- Each iteration still takes **30 cycles**
- Pipelined, a new one starts **every cycle**
  - Total ≈ L + (n − 1)·II
  - 30 + 1023 + overhead = 1,063
- 29× faster, for 3× the arithmetic

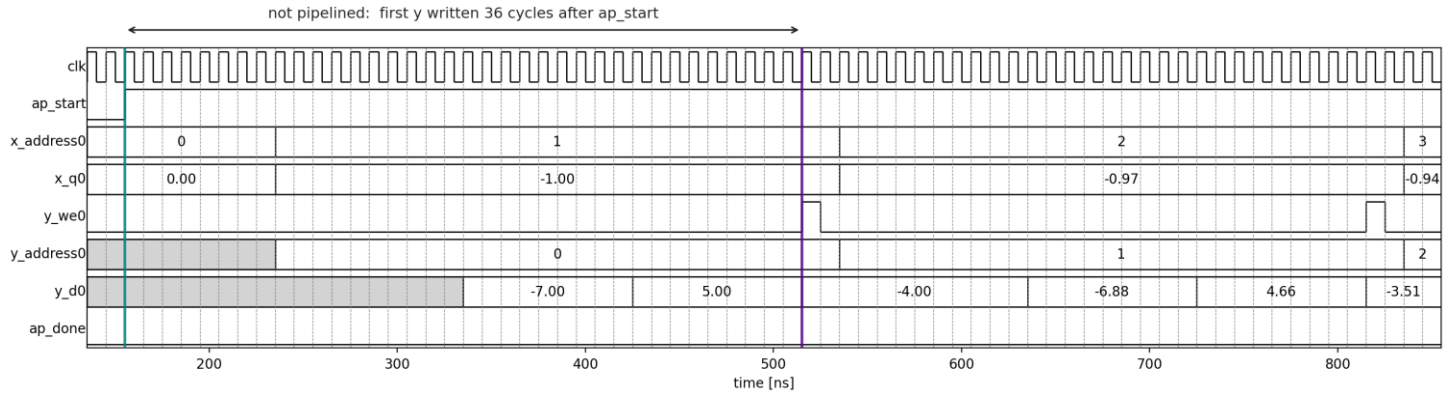
The Pipeline Schedule



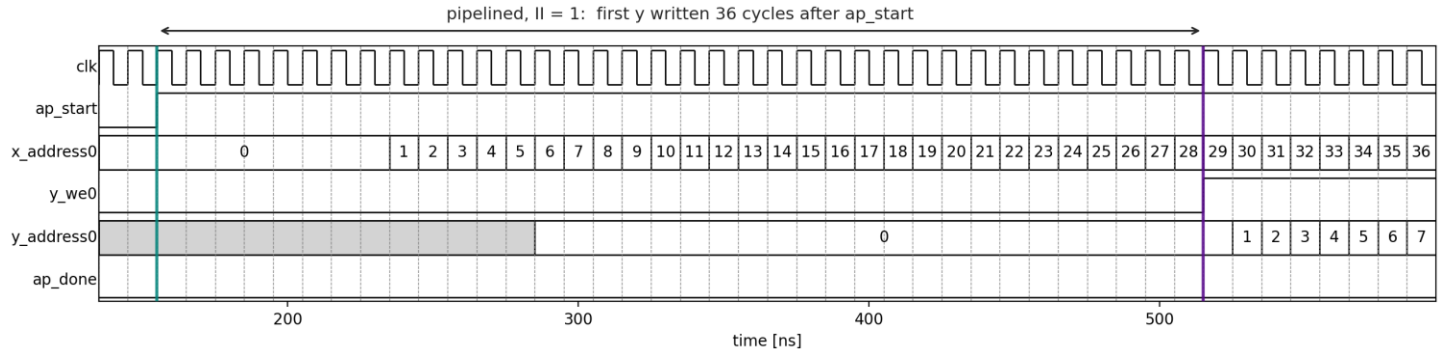
- Top: one iteration, state by state
  - Load 2, then multiply 4 and add 5, three times, then store
  - $2 + 3 \times (4 + 5) + 1 = \mathbf{30}$
- Bottom: iterations start **one cycle apart**
- At cycle 5, iterations 0–3 are all in the first multiply, each at a different stage of it

Measured in Co-Simulation

PIPELINE OFF: ONE y EVERY 30 CYCLES



PIPELINE II=1: ONE y EVERY CYCLE, AFTER 30



- 64 samples, from the RTL simulation's VCD
  - 1,927 cycles off, 99 with the pragma
- The first y takes as long either way
- Then the pipeline delivers one per cycle

Resources and Utilization

	PIPELINE off	PIPELINE II=1
FP multipliers (3 DSPs each)	1	3
FP adders (2 DSPs each)	1	3
DSPs	5	15
Multiplier utilization	12 of 30 cycles: 40%	every cycle: 100%

- Not pipelined: one sample at a time, so **one** multiplier does all three multiplies, one after another
- Pipelined: in every cycle, three multiplies start, **for three samples**
- Each core is itself a pipeline, so it can start one every cycle

The utilization argument again: pipelining keeps the hardware busy, and needs one copy of each operation per II.

Reading the Synthesis Report

PRACTICE

	Design A: off	Design B: II = 1
Function latency, min / max	39 / 30,729	40 / 1,063
Loop iteration latency	30	30
Loop II	–	1
DSP	5	15

The demo's cubic. min is for n = 1 sample, max for n = 1024. Clock 10 ns.

(a) Design A's latency as a formula in n. Which loop entry is its per-sample cost? Cycles and µs for n = 64

(b) The same for design B

(c) The smallest n for which B is at least 10× faster; the speedup for large n

(d) 3 multiplies and 3 adds per sample: the fewest multipliers and adders each design needs. Check against the DSPs (3 per multiplier, 2 per adder)

Also in the LLM grader: Unit 7 › Reading the synthesis report

Reading the Synthesis Report

SOLUTION

(a) (30729 − 39) / 1023 = 30 per sample, the **iteration latency**: **30n + 9**. n = 64: **1,929 cycles, 19.29 µs**

(b) (1063 − 40) / 1023 = 1 per sample, the **II**: **n + 39**. n = 64: **103 cycles, 1.03 µs**

(c) 30n + 9 ≥ 10(n + 39) gives n ≥ 19.05: **n = 20**. For large n the speedup tends to **30**, the ratio of the per-sample costs

(d) A: **1 + 1**, shared, 3 + 2 = 5 DSPs. B: **3 + 3**, all busy every cycle, 9 + 6 = 15 DSPs

Common error

Using the estimated 7.256 ns for the time. The design runs at the 10 ns target; the estimate only says it meets it.

Co-simulation measured 1,927 and 99 cycles at n = 64: within a few cycles of the report.

Also in the LLM grader: Unit 7 › Reading the synthesis report

In Class: One Read per Iteration

IN CLASS

$y[i] = x[i]^2 + x[i-1]^2$, with $x[-1] = 0$. x and y are block RAMs with **one port each**.

The obvious loop asks for II = 1. The report says **II = 2**.

```
for (int i = 0; i < N; i++) {
#pragma HLS PIPELINE II=1
    float prev = (i == 0) ? 0.0f : x[i-1];
    y[i] = x[i]*x[i] + prev*prev;
}
```

(a) How many times does an iteration read x? How many reads can the memory do per cycle?

(b) Each x[k] is read by two iterations. Which?

(c) Rewrite the loop in `sqsum.cpp` to reach **II = 1**. Keep x single-port; y must still match the model

(d) Optional: `--through timing`, and compare the x addresses before and after

sdrangan.github.io/hwdesign — Demos › Loop Optimization › In-Class Exercise

RUN

```
cd demos/loopopt_inclass
python sqsum_build.py
```

Under a minute per build. Only sqsum.cpp is edited; the build checks it, synthesizes it beside the reference and prints both.

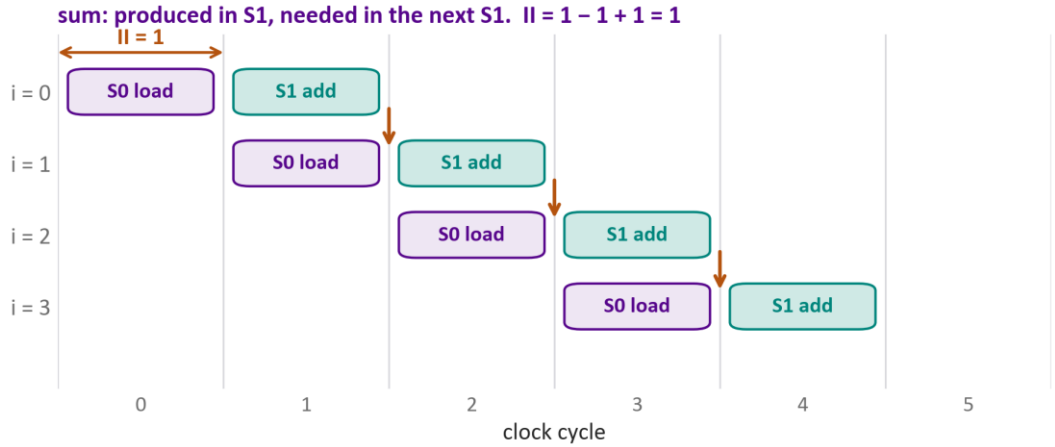
Outline

- Pipelining in SystemVerilog
- Loop Pipelining in Vitis HLS
- Limits on the Initiation Interval

Finding the Minimum II

- **1.** Write one iteration **stage by stage**, one cycle per stage
  - A load takes a stage of its own; a k-cycle operation, k stages
- **2.** For each value carried to the next iteration: the stage S_n that produces it, and the stage S_m of the next iteration that needs it
- **3.** Then **II ≥ n − m + 1**; take the largest
  - Iteration i starts in cycle t: S_n ends at t + n
  - Iteration i + 1's S_m starts at t + II + m, which must be later
  - If n < m, the value is ready in time: no limit beyond II = 1

```
int sum = 0;
for (int i = 0; i < N; i++) {
#pragma HLS PIPELINE II=1
    sum += x[i];
}
// S0: x0 <= x[i]        a load takes a cycle
// S1: sum <= sum + x0   a one-cycle add
```



Loop-Carried Dependencies

- Not every loop pipelines to II = 1
- Example: **merge** two sorted lists x_1 and x_2 into one sorted list y
- Each step
  - Compares the current $x_1[i]$ and $x_2[j]$
  - Writes the smaller to y[k]
  - Advances the index it took from
- Which index advances is decided by **this** iteration's compare

```
i = 0;  j = 0;
for (k = 0; k < N1 + N2; k++) {
    if (i == N1) {            // x1 used up
        y[k] = x2[j];  j++;
    } else if (j == N2) {     // x2 used up
        y[k] = x1[i];  i++;
    } else if (x1[i] < x2[j]) {
        y[k] = x1[i];  i++;
    } else {
        y[k] = x2[j];  j++;
    }
}
```

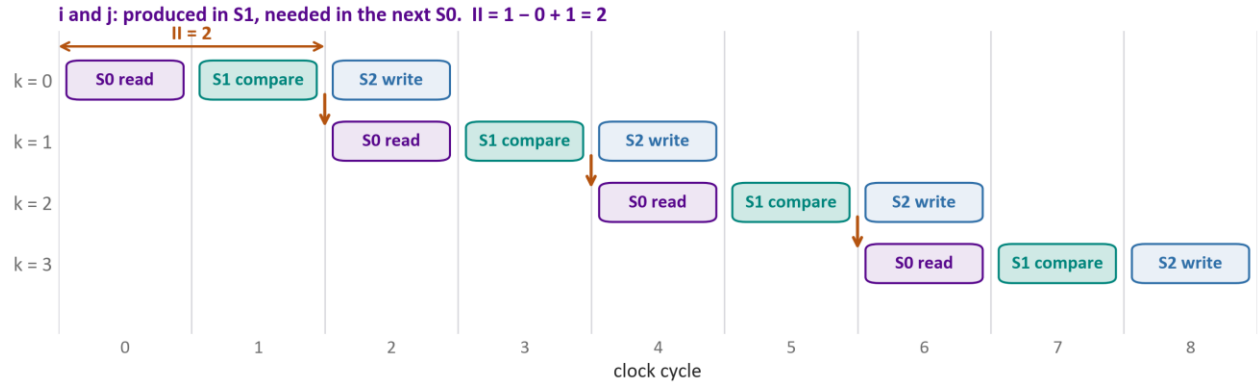
Merge Example

k	i	j	x1[i]	x2[j]	y[k]
0	0	0	1	2	1
1	1	0	4	2	2
2	1	1	4	3	3
3	1	2	4	6	4
4	2	2	10	6	6
5	2	3	10	–	10

- $x_1 = 1, 4, 10$ and $x_2 = 2, 3, 6$
- Each row is one iteration
- i and j advance **unevenly**: which one depends on the data
- At k = 5, x_2 is used up ($j = N_2 = 3$), so y takes $x_1[2]$

The Pipelining Limit

```
// S0: read at i and j
x1i <= x1[i];
x2j <= x2[j];
// S1: compare, and advance i or j
if (x1i < x2j) begin
    yk <= x1i;  i <= i + 1;
end else begin
    yk <= x2j;  j <= j + 1;
end
// S2: write
y[k] <= yk;  k <= k + 1;
```

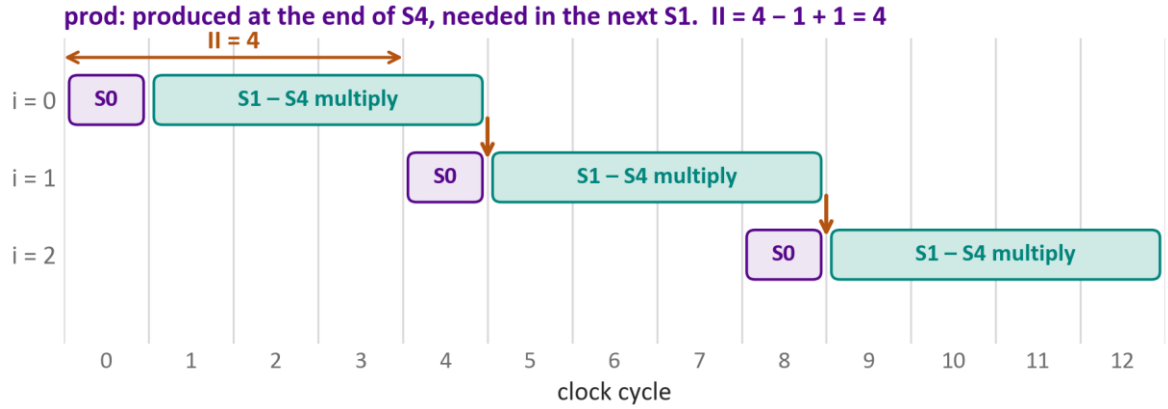


- i and j are **produced in S1** and **needed in S0** of the next iteration
- So **II ≥ 1 − 0 + 1 = 2**: a **loop-carried dependency**

Homework in the LLM grader: Unit 7 › Loop carry dependency

A Multi-Cycle Recurrence

- A float multiply takes several cycles: say **4**
  - S0: load x[i]; S1 to S4: the multiply
- prod is **produced at the end of S4** and **needed in S1** of the next iteration
- So **II ≥ 4 − 1 + 1 = 4**, whatever the pragma asks
  - The multiplier is pipelined, but the next multiply's input is not ready
  - Vitis reports the II it achieved, and warns




```
float prod = 1;
for (int i = 0; i < N; i++) {
#pragma HLS PIPELINE II=1
    prod *= x[i];
}
```

The same rule as before: a multi-cycle operation just spans more stages.

Floating-Point Accumulation

PRACTICE

N = 960. The float adder is 4 stages, f1 to f4, one cycle each: an add that enters f1 in cycle t can feed an add entering f1 in cycle t + 4. A new, independent add can enter every cycle.

(a) Iteration 0's add enters f1 in cycle 0. When can iteration 1's? The II, and the cycles for N = 960

(b) Rewrite with four partial sums, sum0 = x[0] + x[4] + …, sum1 = x[1] + x[5] + …, one updated per iteration. The II and cycles

(c) Tabulate cycles 0–7: the add in each of f1 to f4. When does sum0 + x[4] enter f1, and where does its sum0 come from?

(d) Unroll by 3 instead, into sum0, sum1, sum2 each updated every iteration: the II, the cycles, the fewest adders

```
float sum = 0;
for (int i = 0; i < N; i++) {
#pragma HLS PIPELINE II=1
    sum += x[i];
}
```

Also in the LLM grader: Unit 7 › Floating-point accumulation

Floating-Point Accumulation

SOLUTION

(a) Cycle **4**: it needs the sum iteration 0's add produces. **II = 4**, about 4 × 960 = **3,840 cycles**

(b) Each partial sum is updated every 4th iteration, just as its last add finishes: **II = 1**, about **960 cycles**

(d) Each running sum is still a 4-cycle recurrence: **II = 4**, 320 iterations, about **1,280 cycles**. 3 adds in 4 cycles: **one adder**, not three

```
float part[4] = {0, 0, 0, 0};
#pragma HLS ARRAY_PARTITION variable=part type=complete
for (int i = 0; i < N; i++) {
#pragma HLS PIPELINE II=1
    part[i % 4] += x[i];
}
sum = part[0] + part[1] + part[2] + part[3];
```

Vitis HLS 2025.1 at 100 MHz, a 4-cycle fadd: 3,842 cycles at II = 4; 964 at II = 1 with one adder; 1,284 unrolled by 3. A switch over four scalars instead of the array stays at II = 4: Vitis sees a dependence every iteration.

Also in the LLM grader: Unit 7 › Floating-point accumulation

The Adder Pipeline

SOLUTION

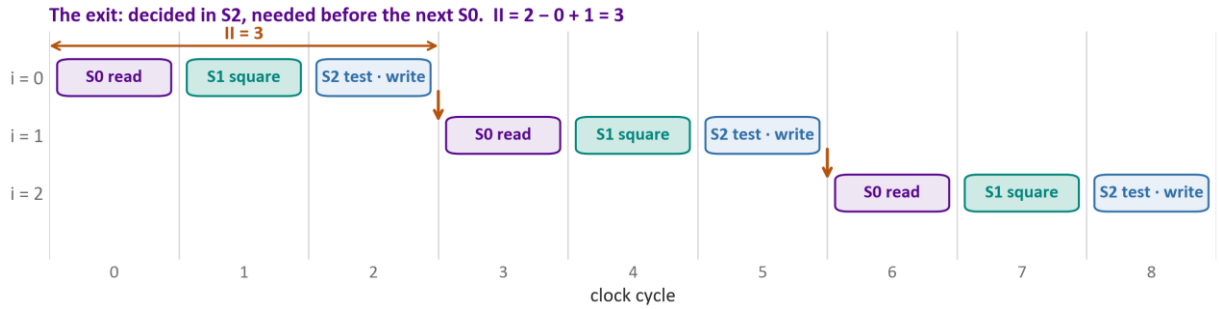
Cycle	f1	f2	f3	f4
0	sum0 + x[0]			
1	sum1 + x[1]	sum0 + x[0]		
2	sum2 + x[2]	sum1 + x[1]	sum0 + x[0]	
3	sum3 + x[3]	sum2 + x[2]	sum1 + x[1]	sum0 + x[0]
4	sum0 + x[4]	sum3 + x[3]	sum2 + x[2]	sum1 + x[1]
5	sum1 + x[5]	sum0 + x[4]	sum3 + x[3]	sum2 + x[2]
6	sum2 + x[6]	sum1 + x[5]	sum0 + x[4]	sum3 + x[3]
7	sum3 + x[7]	sum2 + x[6]	sum1 + x[5]	sum0 + x[4]

- **(c)** sum0 + x[4] enters f1 in **cycle 4**
- Its sum0 is the result of sum0 + x[0], which leaves f4 at the end of cycle 3
- From cycle 3 on, all four stages are busy, each on a different partial sum
- The partial sums **circulate in the adder's own registers**

Also in the LLM grader: Unit 7 › Floating-point accumulation

Early Termination

```
// x: ap_int<16>.  Keep the first NZ
// squares above zmin, then stop.
int j = 0;
for (int i = 0; i < NX; i++) {
#pragma HLS PIPELINE II=1
    int zi = x[i] * x[i];
    if (zi > zmin) {
        z[j] = zi;  j++;
    }
    if (j >= NZ) break;
}
```

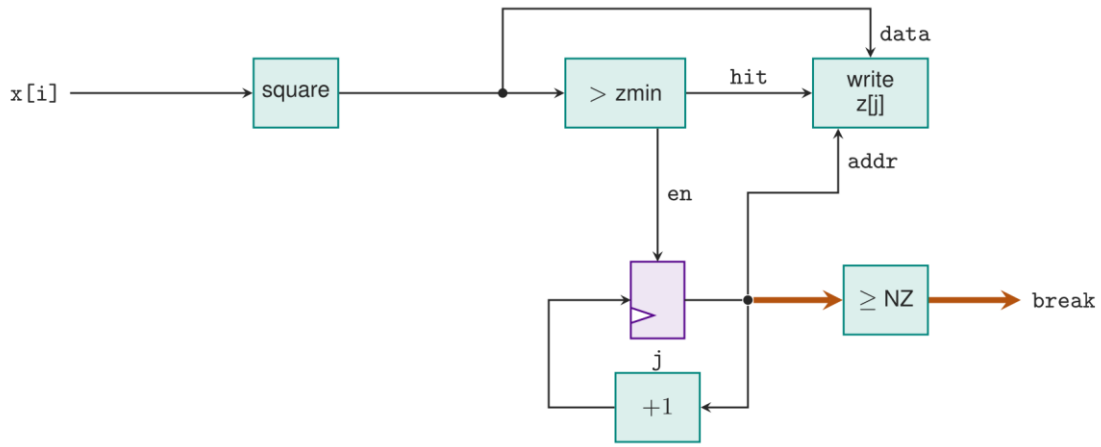


- A break **on data** is a loop-carried dependency
  - A break on a counter, if (i >= n), is decided in S0: no limit

- Iteration i + 1 may start only once iteration i **did not break**
- That needs j, **produced in S2**: so **II ≥ 3**
- Asked for II = 1, Vitis **packs the iteration into one cycle** instead: an estimated **16.3 ns**, at a 10 ns target

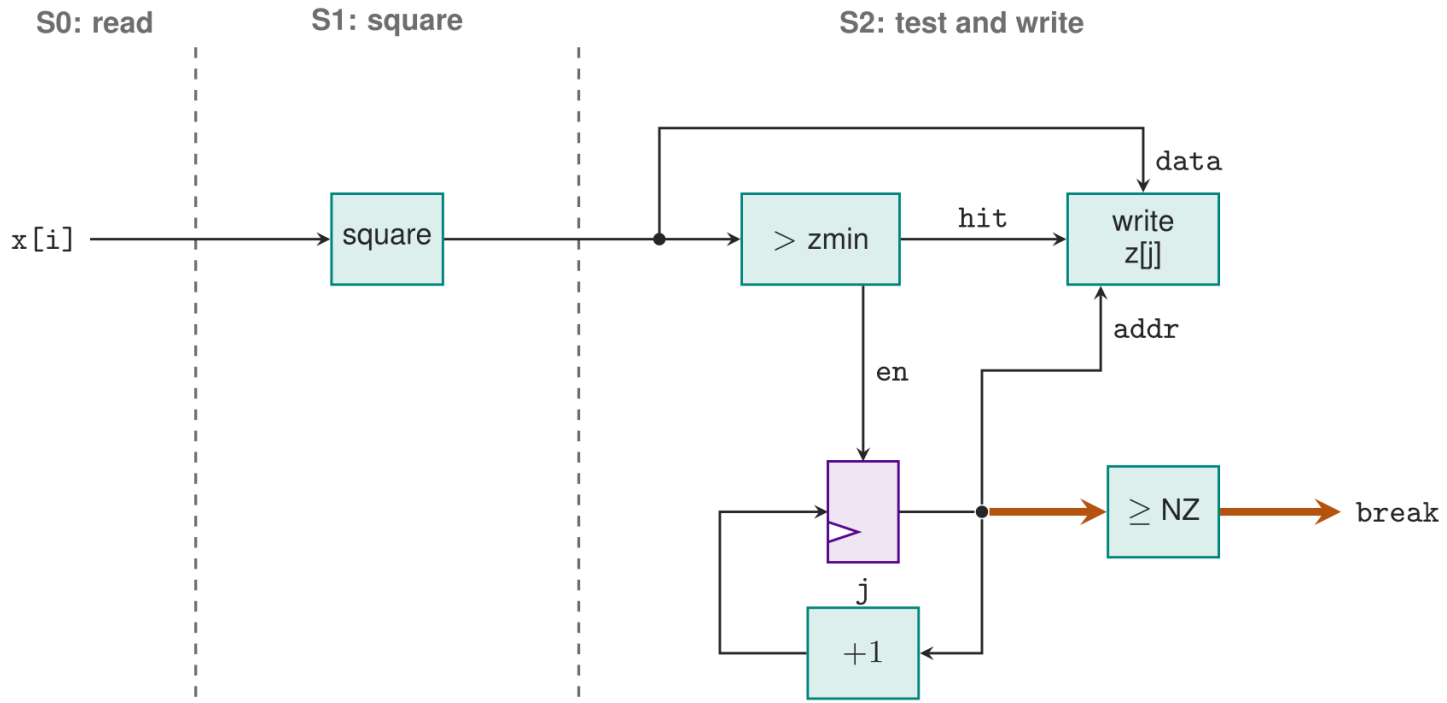
Pipelining by Hand

- **1.** Draw the **computation graph** of one iteration
- **2.** **Cut it into stages**, each one clock cycle
- **3.** Put a **register on every edge that crosses a cut**
  - Name it for the stage that writes it: z_s1 is z, written by S1
  - Pass through what a later stage needs: v_s1 = v_s0
- **4.** Write the body **last stage first**
- **5.** **Fill and drain** with valid bits
  - The same steps as pipelining in SystemVerilog



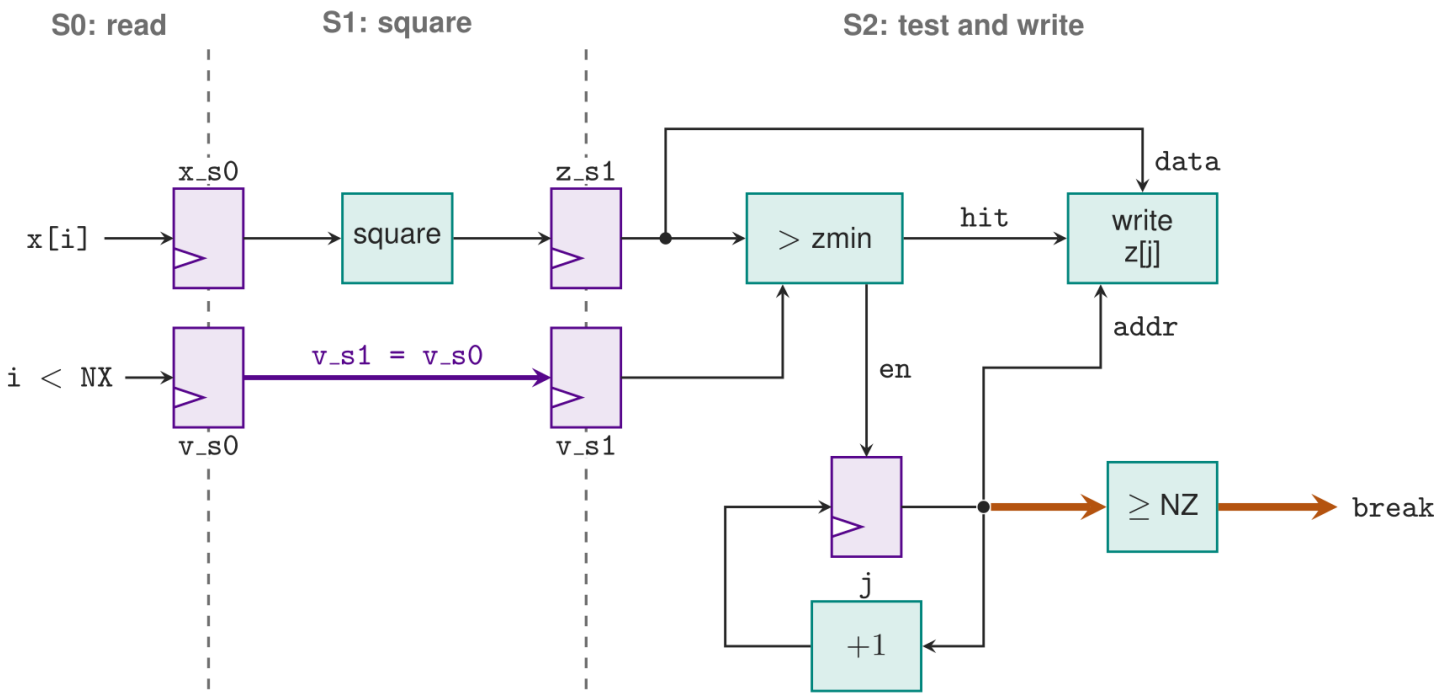
One iteration of the early-exit loop

Cutting into Stages



- **S0** reads, **S1** squares, **S2** tests and writes
- Each stage must **fit one clock cycle**
- j is **produced and needed in S2**
  - II ≥ 2 − 2 + 1 = 1
- So the exit test reads only the **j register**

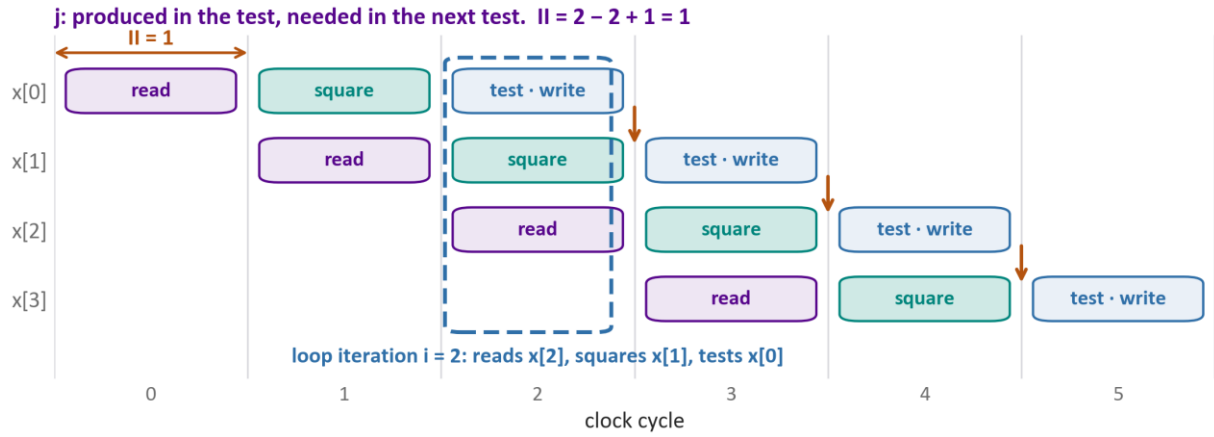
Naming the Registers



- A register wherever a value **crosses a cut**
- Named for the stage that writes it: **x_s0**, **z_s1**
- **Pass through** what a later stage needs: S2 must know z_s1 holds a sample, so **v_s1 = v_s0**
  - Like x1_s1 in the multi-path example
- Data registers need **no reset**; the valid bits do

The Code, Last Stage First

```
ap_int<16> x_s0;  bool v_s0 = false;
int        z_s1;  bool v_s1 = false;
int j = 0;
for (int i = 0; i < NX + 2; i++) {
#pragma HLS PIPELINE II=1
    if (j >= NZ) break;
    // S2: test and write
    if (v_s1 && z_s1 > zmin) {
        z[j] = z_s1;  j++;
    }
    // S1: square
    z_s1 = x_s0 * x_s0;  v_s1 = v_s0;
    // S0: read
    v_s0 = (i < NX);
    if (v_s0) x_s0 = x[i];
}
```



- One block per stage, in **reverse order**
- Each stage reads the _s registers **before** the stage ahead of it overwrites them: what **<=** does in SystemVerilog
- Each iteration works on **three samples**

Vitis HLS 2025.1 at 10 ns: II = 1, an estimated 6.53 ns. C simulation and RTL co-simulation match the loop as written.

Filling and Draining

Iteration	S0 reads	S1 squares	S2 tests
i = 0	x[0]	–	–
i = 1	x[1]	x[0]	–
i = 2	x[2]	x[1]	x[0]
…	…	…	…
i = NX − 1	x[NX−1]	x[NX−2]	x[NX−3]
i = NX	–	x[NX−1]	x[NX−2]
i = NX + 1	–	–	x[NX−1]

- **Fill**: in iterations 0 and 1, z_s1 holds no sample yet. v_s1 is false, so S2 skips the test
- **Drain**: run the loop to **i < NX + 2**, so the last two samples reach S2
- But **guard the read**: v_s0 = (i < NX), and read x[i] only if v_s0
- One loop, no prologue or epilogue: the **valid bits** handle both ends

Also in the LLM grader: Unit 7 › Early exit by hand

Three Ways to Stop

	II	Clock	Cycles	Time
As written	1	16.3 ns ✗	66 – 258	1.1 – 4.2 µs
No break	1	6.8 ns	260	2.6 µs
By hand	1	6.5 ns	66 – 260	0.66 – 2.6 µs

Vitis HLS 2025.1, a 10 ns target. Cycles depend on when z fills: at least NZ = 64 samples.

```
for (int i = 0; i < NX; i++) {
#pragma HLS PIPELINE II=1
    int zi = x[i] * x[i];
    if (zi > zmin && j < NZ) {
        z[j] = zi;  j++;
    }
}
```

No break: stop writing, not looping

A break is free when its test is **cheap and early**. When it needs this iteration's results, either **drop the break** and run every iteration, or **move the work into earlier iterations** so the test reads only registers.